Disaster Intelligence Project

1st Version creation date: 14.08.2026

Data Engineering Aspects and Tasks

First check the availability, accessibility, and format of the following types of data that may need to be downloaded.

S. No.	Data	Sources
1.	Weather and Rainfall	IMD open data, MOSDAC
2.	Satellite Imagery	Sentinel 2, LandSat 8/9, MODIS Terra/Aqua (Daily feeds), VIIRS (Daily feeds), ASTER (On demand). Also collect SAR satellite data like Sentinel 1, ALOS, PALSAR, NISAR, SAOCOM
3.	Digital Elevation Maps (DEM) and Terrain	Carto DEM (India Specific), Global like SRTM, CopernicusDEM, ASTERDEM
4.	Environmental Data	Copernicus Landcover, ESA world cover, Soil Grids, FAO Soil map
5.	Hazard Maps	Geological Survey of India, National Landslide Susceptibility maps, Bhuvan, NRSC, NDMA
6.	Earthquake events	National Center for Seismology (India), IRIS, ISC, USGS Earthquake catalog
7.	Landslide inventories	
8.	Hydrology	India-WRIS, CWC hydrological data
9.	Social media benchmark dataset	Crisis-MMD, Twitter-API, Reddit-API, Youtube-API, Flickr, Telegram public channel
10.	News feeds	Google news RSS, PIB, NDMA bulletins
11.	Historical disaster reports	
12.	Infrastructure and Population	PM Gati Shakti, India Census, Landscan
13.	GIS open street map	Microsoft Building Footprints, Google Open Buildings, Natural Earth
14.	UAV Data	XBD, Floodnet, LoveDA, VAVid, DOTA, VisDrone, Aerial Image Dataset (AID)

Do not start with large scale data downloading now. We first need to fix the following before data collection happens in the databases:

1. Canonical multimodal data schema
2. Disaster/Ontology Knowledge graph schema
3. Spatial-temporal event linking strategy

Step 1:

1. Create object store for raw data collection. Maintain an immutable raw version of the data collected. Use an S3-compatible object store. As we would be using our own server and not

cloud computation, preferably use implement MinIO on your own server. We would install the MinIO within a Docker container so that it can be made to function independently, and is not affected by the existing software and applications running in the system. Study a bit about docker containers and MinIO, before implementing the same. Example structure could be:

```
/raw/
  weather/
    era5/
    gpm/
    imd/
  satellite/
    sentinel1/
    sentinel2/
    landsat/
  social/
  news/
  sensors/
  earthquake/
  landslide/
  gis/
```

The raw object data store should be immutable and in no situation the raw data collected should be over written. We need to create a separate working data store on top of the raw object store for further functionalities. The data that we would be working with are of different types and require different storage capabilities. For example, satellite and UAV data need to support spatial queries, like showing damages within 2kms from a location X, sensor data need to support temporal queries, like all incidents from 2 to 5pm on a given date. Handling these specific types of queries require different data storage and processing capabilities, which are not provided by a single database. So we would use a polyglot architecture, where we use different databases for specific types of data, and they would be linked via common attributes, like event_id, location_id etc. At the core, the metadata information of each of these data would be stored in a PostgreSQL database and the spatial data in a PostGIS database. The temporal data would be stored in a TimescaleDB database that sits on top of the PostgreSQL and PostGIS databases. These 3 databases together would form our core operational database. Apart from these we would be using OpenSearch for handling textual data like social media, news contents, reports and textual contents from any other source. We would also use a Neo4j database to store a knowledge graph derived from these databases. We next need to design the schema for each of these databases.

Step 2:

1. Schema building for core metadata and spatial database. Do not put satellite images or large NetCDF files into any relational database. We would use PostgreSQL and PostGIS as the central database for this purpose. The database may contain the following canonical entities tentatively, illustrated for the purpose of giving a broad idea. Rather that a single table we will use a master relational model like as follows:

   DataSource(

source_id
source_name
source_type
organization
license
refresh_frequency
api_endpoint
contact)

Examples may include ERA5, GPM, Sentinel2, Twitter etc.

Observation(
observation_id
source_id
modality
observation_type
time_start
time_end
geometry
location_id
event_id
quality_score
confidence
raw_object_uri
processed_object_uri
status)
Location(
location_id
latitude
longitude
district
state
country
watershed
village
elevation
geom
)

Event(
event_id
event_name
hazard_type
start_time
end_time
severity
status
description

)

Hazard(
hazard_id
hazard_name
parent_hazard
)

AdministrativeRegion(
region_id
district
state
country
polygon
)

Asset(
asset_id
asset_type
location_id
name
owner
status
geometry
)

Prediction(
prediction_id
model_name
event_id
location_id
prediction_time
risk_score
confidence
)

RainfallObservation(
rainfall_id
observation_id
rainfall_mm
temperature
humidity
pressure
wind
)

SatelliteScene(
scene_id

observation_id
satellite
sensor
bands
resolution
cloud_cover
)
This now becomes the structured layer. The schemas shown here are not exhaustive and is outlined for illustration only. You need to come up with the entire set of schemas. Study how to create good schema designs. We will need to discuss on these elaborately.

2. Temporal or Time series data. When we collect real sensor data, the sensor data becomes the dominant workload. For handing temporal data, we would use TimeScaleDB over PostgreSQL. Do not introduce any new separate database. You may populate the temporal database using simulated sensor data. Create a common canonical schema for the sensor data like:
SensorObservation(
observation_id
source_id
modality
observation_type

valid_time_start
valid_time_end

observation_time
ingestion_time

geometry
location_id
event_id

quality_score
confidence

raw_object_uri
processed_object_uri
)

For rainfall or any other specific sensor data we can have a separate schema connected by a foreign key like

RainfallObservation(
rainfall_id
observation_id
rainfall_mm
temperature
humidity
pressure

wind
)
Then populate it initially using data like public weather stations, GPM rainfall pixels, river gauges, earthquake stations, synthetic sensors sampled from public datasets. Later we may use the same structure for the collected data.

So the schema would look something like this:

Common Linking Layer Schema (Layer 1)							
Data_id	source	time	location	Event_id	modality	quality	Provenance
Layer 2							
Weather schema		Social Media Schema		Satellite Schema		Sensors Schema	

3. Shared ontology creation. You have to create a shared ontology that will be used to create knowledge graphs. Examples like this as follows:

Hazard
|- Landslide
|- Flood
|- Earthquake

Impact
|- RoadDamage
|- BuildingDamage
|- Casualty
|- PowerFailure

Need
|- Food
|- Water
|- Shelter
|- MedicalAid
|- Rescue

Step 3:

1. Knowledge Graph Creation. Use Neo4j to represent the knowledge graphs. However, the relations have to be predefined, and may require machine learning techniques as well. You need to come up with the list of relations and methods for extracting the relations. Example Knowledge graph looks as follows:

(Landslide)-[:OCCURRED_AT]->(Joshimath)

(Landslide)-[:TRIGGERED_BY]->(HeavyRainfall)

(RainfallObservation)-[:OBSERVED_AT]->(Location)

(SentinelImage)-[:COVERS]->(Location)

(Tweet)-[:REPORTS]->(Landslide)

(Landslide)-[:DAMAGED]->(Road)

(Village)-[:EXPOSED_TO]->(LandslideRiskZone)

(Shelter)-[:SERVES]->(AffectedPopulation)

2. Search and semantic retrieval framework. Need to use OpenSearch for search over news, social media, text and multimodal embedding. OpenSearch supports both keyword based as well as vector based searches.

So, as stated earlier we use a polyglot architecture that uses different kinds of databases/storage systems for different kinds of data, rather than forcing everything into one database. For our project this is necessary because the data types are fundamentally different. Satellite imagery is large binary/raster data, sensor feeds are high-frequency time series, GIS layers are spatial, social media and reports are text-heavy, and the knowledge base is graph-structured. So instead of everything into one database we have

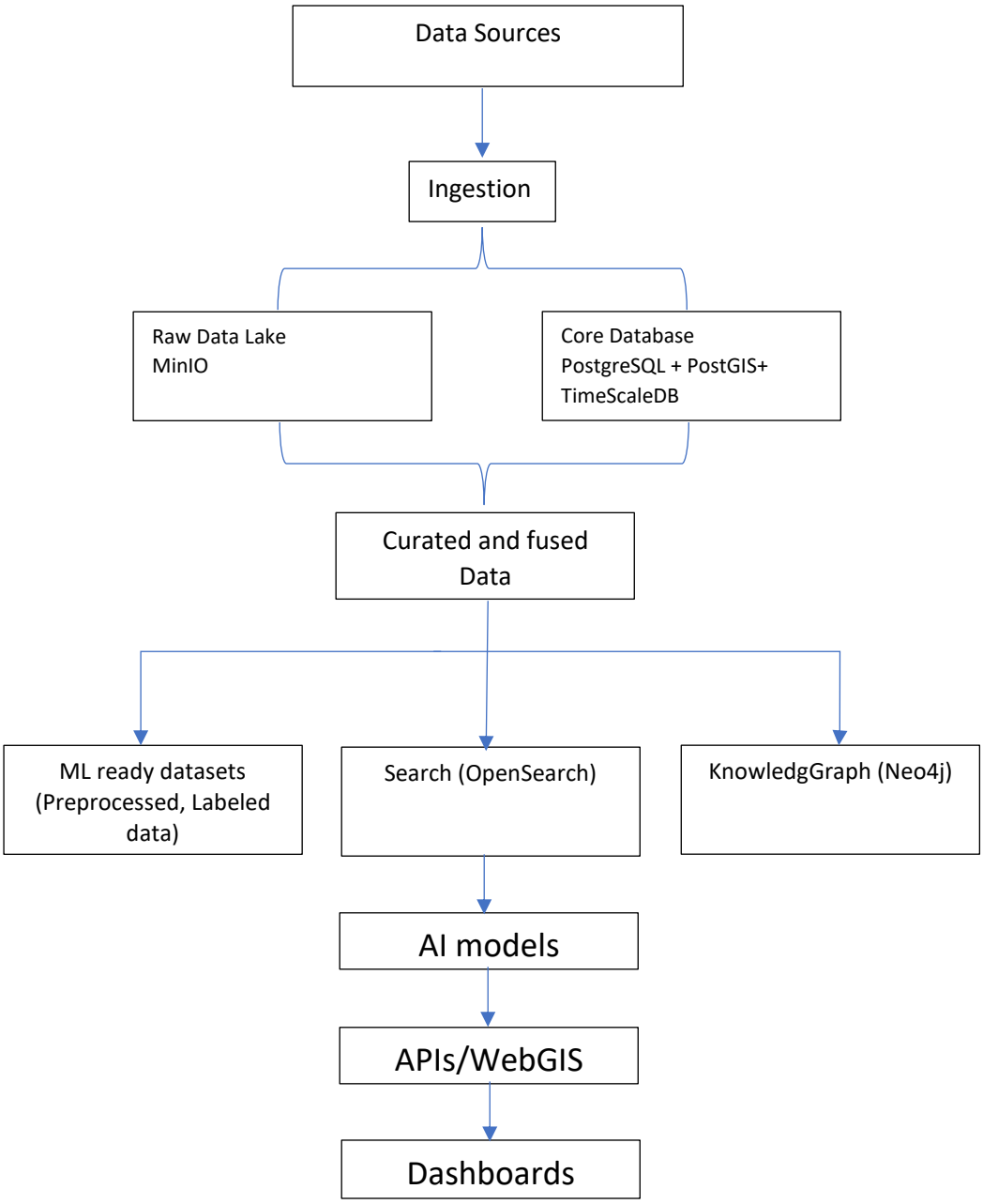
Satellite and large files → stored in → Object store / MinIO
GIS + core metadata → PostgreSQL + PostGIS
Sensor / weather time series → TimescaleDB
Text / semantic search → OpenSearch
Knowledge graph → Neo4j

The important thing is that the databases are not independent of each other, rather they are connected through common identifiers like observation_id, event_id, location_id, source_id, timestamp or time_interval etc. Thus PostgreSQL + PostGIS + TimescaleDB acts as the main operational database.

Step 4:

1. AI based fusion models. Once we have the ML ready data in the databases, the Knowledge graph built using Neo4j and the text data loaded and indexed using the Opensearch platform, we need to perform cross-modal fusion of the data to run AI based models for downstream tasks and dashboard based visualization.
2. WebGIS and API development. The dashboard visualizers are not going to interact with the ML models and databases directly. Rather they need to use API calls like GET to obtain the data. Hence appropriate APIs need to be built.

So the architecture looks something like this:


Data Sources
Ingestion
Raw Data Lake
MinIO
Core Database
PostgreSQL + PostGIS+
TimeScaleDB
Curated and fused
Data
ML ready datasets
(Preprocessed, Labeled
data)
Search (OpenSearch)
KnowledgGraph (Neo4j)
AI models
APIs/WebGIS
Dashboards